
PROJECT REPORT



Title: Traffic Flow Optimization System

Difficulty Level	Full
Programming Language	C++
Course Name	Data Structures
Instructor	Dr. Anum Abdul Salam
Institution	NUST CEME, Rawalpindi
Submission Date	4 May 2026

Submitted By

Sr. No.	Student Name	CMS ID
1	Azeem Ashraf	504116
2	Muhammad Umer Shahzad	507904
3	Hafiz Muzammil Hussain	533058

**Department of Computer and Software Engineering,
National University of Science and Technology (NUST), Islamabad**

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Background of the Application Domain	4
1.2 Motivation for Choosing the Problem	4
1.3 Real-World Relevance	4
1.4 Importance of Simulation	4
1.5 Overview of the Proposed System	4
2. Problem Statement	5
2.1 Clear Definition of the Problem	5
2.2 Limitations of Existing Approaches	5
2.3 Challenges in Real-World Implementation	5
2.4 Justification for Simulation	5
3. Objectives	6
3.1 Primary Objective	6
3.2 Secondary Objectives	6
3.3 Learning Outcomes	6
3.4 Expected System Capabilities	6
4. System Overview	7
4.1 High-Level Description	7
4.2 Overall Workflow	7
4.3 Input Sources	7
4.4 Expected Outputs	7
5. System Design	8
5.1 Overall System Architecture	8
5.2 Modular and Object-Oriented Design	8
5.3 Interaction Between Major Components	8
6. UML Class Diagram	9
7. System Level Diagram	10
8. Mathematical / Logical Modeling	11
8.1 Graph Representation of Road Network	11
8.2 Traffic Flow Model	11
8.3 Queue Formation Model	11
8.4 Congestion Model	12

8.5 Travel Time Function (BPR Formula).....	12
8.6 Vehicle State Transition.....	12
8.7 Routing Using Dijkstra's Algorithm	12
8.8 Traffic Signal Decision Logic.....	12
8.9 Objective Function.....	13
9. Time Complexity Analysis	13
10. Implementation Details.....	14
10.1 Programming Language and Tools	14
10.2 Project File Structure	14
10.3 Key Classes and Methods	15
11. Key Features	15
12. Results and Outputs	16
12.1 Description of Outputs.....	16
12.2 GUI Screenshot.....	16
12.3 Observed Behavior.....	17
12.4 Performance Trends	17
13. Challenges and Solutions.....	17
13.1 Technical Challenges	17
13.2 Design Challenges	17
13.3 Implementation Issues	18
14. Conclusion	18
14.1 Summary of Achievements.....	18
14.2 Objective Fulfillment.....	18
14.3 Learning Outcomes.....	18
15. Future Enhancements.....	20
15.1 Possible System Extensions.....	20
15.2 Additional Features	20
15.3 Real-World Deployment Possibilities.....	20
16. References.....	20
Books	20
Online Resources	20

1. Introduction

1.1 Background of the Application Domain

Modern cities face serious traffic congestion problems due to increasing numbers of vehicles and outdated traffic control systems. Traditional traffic management uses fixed signal timings and static routing, which cannot adapt to real-time traffic conditions. This leads to long delays, wasted fuel, and poor road utilization. The need for smarter, data-driven traffic management systems has grown significantly over the past decade.

1.2 Motivation for Choosing the Problem

This project was chosen because traffic optimization is a real and pressing problem that directly affects people every day. It is also an excellent application domain for data structures and algorithms, graphs, queues, shortest path algorithms, and observer patterns all have direct practical use here. Building a working simulation of this system demonstrates how theoretical computer science concepts solve real-world problems.

1.3 Real-World Relevance

Road networks naturally form graphs, intersections are nodes and roads are edges. Every city navigation system like Google Maps uses variants of the algorithms implemented in this project. Adaptive traffic signal control systems used in modern smart cities use the same congestion modeling and queue management principles designed in this simulation.

1.4 Importance of Simulation

A simulation-based approach allows safe and cost-effective testing of traffic optimization strategies without risking real road infrastructure. It allows evaluation of different routing strategies, signal timing policies, and congestion scenarios under controlled conditions. The discrete-time simulation loop used in this project closely mirrors how real traffic simulation software operates.

1.5 Overview of the Proposed System

The Traffic Flow Optimization System models a city road network as a directed weighted graph. Vehicles are generated at source intersections and routed to destinations using Dijkstra's shortest path algorithm, where edge weights represent real-time travel times calculated using the Bureau of Public Roads (BPR) congestion function. A Windows Forms GUI with a background simulation thread provides a live visual representation of the entire simulation.

2. Problem Statement

2.1 Clear Definition of the Problem

The objective is to design and implement a simulation-based system that models an urban road network as a directed graph $G = (V, E)$, simulates vehicle movement and traffic flow, models congestion and queue formation at intersections, dynamically adjusts traffic signal timings, computes optimal routes for vehicles using current traffic states, and minimizes congestion and travel time across the entire network.

2.2 Limitations of Existing Approaches

Traditional traffic management systems rely on static signal timings that are pre-programmed and do not respond to actual traffic conditions. Fixed routing does not account for real-time congestion on roads. These systems fail in high-traffic scenarios and cannot adapt when roads are blocked or heavily congested. They also provide no mechanism to reroute vehicles dynamically once a route has been assigned.

2.3 Challenges in Real-World Implementation

- Modeling congestion accurately: Travel time on a road increases non-linearly as vehicles approach capacity.
- Queue management at intersections: Vehicles finishing a road must wait in queues before proceeding.
- Signal coordination: Only one incoming road can have a green signal at a time per intersection.
- Dynamic rerouting: When congestion changes, all active vehicles must be rerouted using updated edge weights.
- Thread safety: The simulation runs in a background thread while the GUI renders in the main thread, requiring proper synchronization.

2.4 Justification for Simulation

A simulation allows all these components to be tested together in a controlled environment. The discrete-time tick-based simulation loop enables precise control over the order of updates, flow calculations, signal switching, vehicle movement, and Dijkstra rerouting all happen in a defined sequence every 100ms.

3. Objectives

3.1 Primary Objective

To design and implement a complete Traffic Flow Optimization System in C++ that models an urban road network as a directed weighted graph, simulates vehicle movement and congestion, dynamically adjusts traffic signals, routes vehicles using shortest path algorithms, and visualizes the entire simulation in real time through a GUI.

3.2 Secondary Objectives

- Implement the Observer design pattern to decouple the simulation backend from the GUI.
- Use a custom Graph ADT with adjacency list representation to model the road network.
- Implement a custom Queue ADT for vehicle queue management at intersections.
- Apply the BPR (Bureau of Public Roads) travel time formula for congestion-aware routing.
- Implement adaptive signal control that gives priority to the road with the longest queue.
- Use background threading to eliminate GUI jitter and ensure smooth simulation.
- Provide a 6x6 predefined city grid with full bidirectional roads and interactive editing.

3.3 Learning Outcomes

- Practical application of Graph ADT and Dijkstra's Algorithm to a real problem.
- Implementation of the Observer design pattern in C++.
- Queue-based data structure usage for intersection management.
- Understanding of BPR traffic flow theory and discrete-time simulation.
- Multi-threaded programming with synchronization in C++/CLI Windows Forms.

3.4 Expected System Capabilities

- Model a 6x6 city grid with 36 intersections and 60 bidirectional roads (120 directed edges).
- Simulate multiple vehicles moving simultaneously with real-time congestion effects.
- Dynamically reroute vehicles when road congestion changes.
- Display color-coded roads (gray = free, orange = moderate, red = congested).
- Show traffic light states (green/yellow/red) at each intersection.
- Support user-controlled vehicle spawning, road editing, and intersection parameter changes.

4. System Overview

4.1 High-Level Description

The system is designed as a modular architecture where each class represents a specific physical or computational component. The road network is a directed weighted graph. Roads are edges carrying congestion data. Intersections are nodes that control signal timing. Vehicles are agents moving along routes computed by Dijkstra's algorithm. The Manager class orchestrates the entire simulation tick-by-tick, while the GUI renders the state of the system every 100ms.

The system uses the Observer design pattern throughout: Roads and Intersections are Subjects; the GUI and Manager are Observers. When road congestion changes, the Manager automatically reruns Dijkstra and updates vehicle routes, while the GUI redraws the affected road with its new congestion color.

4.2 Overall Workflow

1. **Initialization:** Initializer builds the 6x6 city grid, 36 intersections and 120 directed edges. **Manager** takes ownership and subscribes itself and the GUI as observers to all roads.
2. **Simulation Loop (every 100ms):** A background thread calls `Manager::tick()`, which in order: syncs graph edge weights with real-time travel times, decrements vehicle remaining times, auto-spawns new vehicles, updates intersection signals, updates road flow and congestion, recalculates performance metrics, and computes the objective function value.
3. **Observer Notification:** When any road updates, it notifies all observers. Manager reroutes vehicles using updated Dijkstra weights.
4. **User Interaction:** User can add vehicles, edit road properties, change intersection **Tg** values, and control simulation speed through the side panel.

4.3 Input Sources

- Predefined city map: 36 intersections at fixed pixel coordinates, all roads at 50 km/h with capacity 15.
- User inputs: source/destination for vehicle, road length/speed/capacity/discharge rate, intersection **Tg**, new intersections.
- Simulation parameters: speed multiplier (x1 to x5) via trackbar.

4.4 Expected Outputs

- Visual city map with color-coded roads, moving vehicles, and traffic light states.
- Real-time performance metrics: Average Travel Time, Total Delay, Throughput, Average Congestion, Objective Value.

- Vehicle tooltip on hover: ID, status, source, destination, remaining time.
- Tick counter showing simulation progress.

5. System Design

5.1 Overall System Architecture

The system is organized into four layers. The Core Patterns layer contains Observer and Subject base classes that enable the notification mechanism. The Simulation Entities layer contains Road and Intersection, both inheriting from Subject. The System Control layer contains Manager (inheriting Observer) and Initializer. The GUI Layer contains the trafficSimulation Windows Form and GUIObserver bridge class. The Data Structures layer provides the custom Graph (AdjList), Queue, and Vehicle components.

5.2 Modular and Object-Oriented Design

Every class has a single responsibility. Subject manages the observer list. Road manages its own congestion, queue, and signal state. Intersection manages signal timing and switching logic. Manager drives the simulation and owns all global state. Initializer builds the network and hands it to Manager. GUIObserver bridges the native C++ Observer interface to the managed Windows Forms class. This design means the backend has zero dependency on the GUI, it can run standalone or with any observer attached.

5.3 Interaction Between Major Components

The Initializer creates all Intersection and Road objects and builds the directed Graph. Manager takes the Initializer and preloads incoming road lists for each intersection. Every 100ms the background thread calls Manager::tick(). Inside tick(), all roads call their own tick() which updates congestion and notifies observers. The GUI observer safely calls BeginInvoke to trigger a panel repaint on the UI thread. When Manager::update() is called as an observer, it syncs graph weights and reroutes all vehicles.

6. UML Class Diagram

Figure 5.1 shows the complete UML class diagram for the Traffic Flow Optimization System. The diagram is organized into five groups: GUI Layer, Core Patterns, System Control, Simulation Entities, and Data Structures.

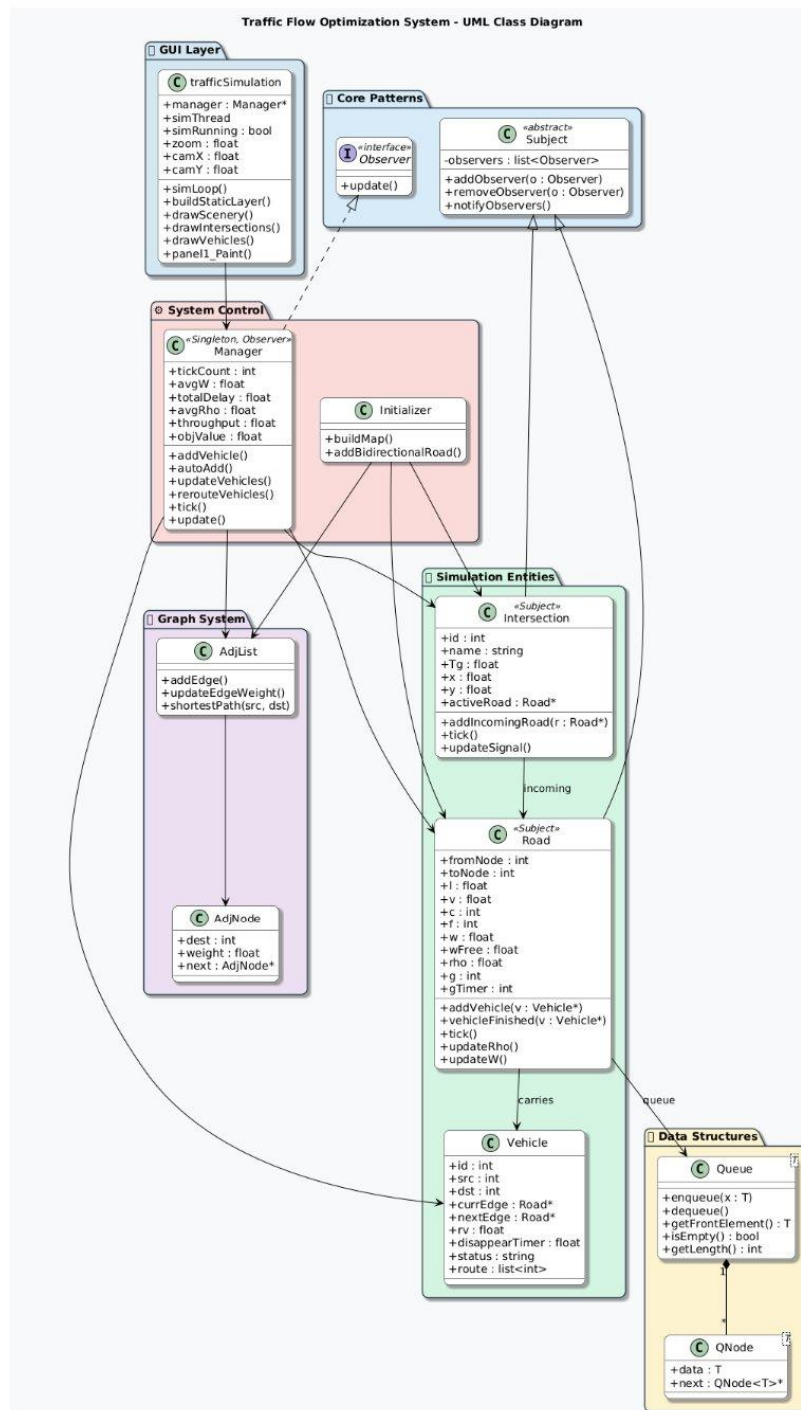


Figure 5.1: UML Class Diagram

The major classes shown in the diagram and their responsibilities are described below.

trafficSimulation	Windows Forms GUI class. Renders city panel, handles user input, subscribes to backend via GUIObserver.
Manager	Drives simulation tick loop. Implements Observer to reroute vehicles when congestion changes. Owns all vehicles.
Initializer	Builds the 6x6 city grid. Adds intersections and bidirectional roads. Hands graph to Manager.
Intersection	Graph node. Inherits Subject. Controls signal timing with green/yellow/red phases. Notifies GUI on change.
Road	Graph edge. Inherits Subject. Maintains flow, congestion, travel time, and vehicle queue. Notifies on change.
Vehicle	Data struct. Holds id, source, destination, current road, remaining time, route, and status.
AdjList	Custom graph ADT using adjacency list. Stores float edge weights for real-time travel times. Implements Dijkstra.
Queue	Custom queue ADT. Used by Road to manage waiting vehicles at downstream intersection.
Observer	Pure abstract interface with update() method. Implemented by Manager and GUIObserver.
Subject	Abstract base with observer list. Provides addObserver, removeObserver, notifyObservers.

7. System Level Diagram

The system is divided into the following subsystems that interact through the Observer pattern and direct function calls. The main execution flow begins with Initializer building the network, Manager taking control, and the GUI rendering the state on every tick notification.

Subsystem Interaction:

- **Initializer:** Constructs all graph nodes (Intersections) and edges (Roads). One-time setup at startup or reset.
- **Manager (Simulation Engine):** Drives the tick loop. Calls syncWeights, updateVehicles, autoAdd, intersection ticks, road ticks, updateMetrics, calcObjective every 100ms.
- **Graph System (AdjList):** Stores the directed road network. Edge weights are updated every tick to reflect real-time travel times. Dijkstra runs on demand for vehicle routing.
- **Road Subsystem:** Each road independently updates its own flow, congestion, and travel time, then notifies observers.
- **Intersection Subsystem:** Each intersection counts down its signal timer and switches the green signal to the road with the longest queue, passing through a yellow phase.

- GUI Subsystem: Receives notifications via GUIObserver. Repaints the city panel using a static cached layer for roads and a dynamic layer for vehicles and signals.

Data Flow:

- Vehicle positions flow from Manager::vehicles list to GUI drawing functions.
- Road congestion data flows from Road::rho to GUI road color and Manager rerouting decisions.
- Graph edge weights flow from Road::w to AdjList every tick before Dijkstra is called.
- Performance metrics flow from Manager calculations to GUI labels on every tick.

8. Mathematical / Logical Modeling

8.1 Graph Representation of Road Network

The road network is modeled as a directed weighted graph:

$$G = (V, E)$$

Where V is the set of intersections (nodes) and E is the set of roads (directed edges). Each edge e_{ij} represents a one-way road from intersection i to intersection j . Since all roads in the predefined city are bidirectional, each physical road is represented as two directed edges — one in each direction — with independent flow, congestion, and queue states.

8.2 Traffic Flow Model

Traffic flow on each road changes over time according to:

$$f_{ij}(t+1) = f_{ij}(t) + a_{ij}(t) - x_{ij}(t)$$

Where $f_{ij}(t)$ is the number of vehicles currently on road (i,j) at time t , $a_{ij}(t)$ is vehicles entering the road, and $x_{ij}(t)$ is vehicles completing the road and joining the downstream queue Q_{ij} .

8.3 Queue Formation Model

When a vehicle completes traversing road (i,j) , it joins the queue at intersection j :

$$Q_{ij}(t+1) = Q_{ij}(t) + x_{ij}(t) - d_{ij}(t)$$

The discharge $d_{ij}(t)$ is governed by the signal state and downstream capacity:

$$d_{ij}(t) = g_{ij}(t) * \min(Q_{ij}(t), \mu_{ij}, c_{jk} - f_{jk}(t))$$

Where $g_{ij}(t)$ is 1 if signal is green, μ_{ij} is the maximum discharge rate, and $c_{jk} - f_{jk}(t)$ is available capacity on the next road.

8.4 Congestion Model

Congestion level is defined as the ratio of current flow to road capacity:

$$\rho_{ij} = f_{ij} / c_{ij}$$

A value of rho near 0 indicates a free road, while rho near 1 or above indicates a congested road. This value drives road color in the GUI and affects travel time calculation.

8.5 Travel Time Function (BPR Formula)

Travel time increases non-linearly with congestion using the Bureau of Public Roads formula:

$$w_{ij}(t) = w_{ij_free} * (1 + 0.15 * (f_{ij}/c_{ij})^4)$$

Where $w_{ij_free} = l_{ij} / v_{ij_max}$ is the free-flow travel time (road length divided by maximum speed), 0.15 is the congestion sensitivity parameter alpha, and 4 is the nonlinearity parameter beta. This formula is the standard BPR function used in transportation network analysis.

8.6 Vehicle State Transition

Each vehicle has a remaining travel time $r_v(t)$ that decrements every tick:

$$r_v(t+1) = r_v(t) - 1$$

When $r_v(t) = 0$, the vehicle has finished the current road and moves to the downstream queue. When the vehicle is discharged from the queue (signal is green and downstream road has capacity), it moves to the next road and r_v is set to the new road's current travel time $w_{ij}(t)$.

8.7 Routing Using Dijkstra's Algorithm

Vehicles select routes dynamically based on current traffic conditions. The edge cost function is:

$$\text{cost}(e_{ij}) = w_{ij}(t)$$

The optimal path P_v for vehicle v from its current node to destination d is:

$$P_v = \text{argmin} \sum(w_{ij}(t))$$

This is computed using Dijkstra's algorithm. Graph edge weights are updated every tick to reflect current travel times before Dijkstra runs, ensuring routes always use the most current congestion data.

8.8 Traffic Signal Decision Logic

Each intersection assigns the green signal to the incoming road with the maximum queue size:

$$g_{ij}(t) = 1 \text{ for edge with maximum } Q_{ij}(t)$$

Signal constraint: Only one incoming road can be green at any time. The system enforces this by first setting all incoming road signals to red, then setting only the road with the longest queue to green. A yellow phase of 3 ticks separates green phases.

8.9 Objective Function

The system minimizes the following objective function at each tick:

$$\min \sum_t \sum_{(i,j)} [\alpha * Q_{ij}(t) + \beta * (f_{ij}(t)/c_{ij})^2]$$

With $\alpha = 0.5$ and $\beta = 0.5$, this function equally weights queue lengths (waiting time) and congestion levels (road utilization). A lower objective value means the simulation is achieving better traffic flow optimization.

9. Time Complexity Analysis

The following table presents the time complexity of all major algorithms and functions in the system. V is the number of intersections (nodes), E is the number of roads (directed edges), and N is the number of active vehicles.

Algorithm / Function	Worst Case	Average Case	Notes
Dijkstra's Algorithm (graph)	$O(V^2 + E)$	$O(V^2 + E)$	Linear scan for min — no priority queue
syncWeights() — update all edges	$O(E)$	$O(E)$	One pass over all roads
updateVehicles() — tick all vehicles	$O(N)$	$O(N)$	One pass over vehicle list
rerouteVehicles() — reroute all	$O(N * (V^2 + E))$	$O(N * V^2)$	Dijkstra per active vehicle
Road::tick() — one road update	$O(\mu)$	$O(1)$	μ = max discharge, small constant
Intersection::tick() — signal update	$O(\text{in-degree})$	$O(\text{in-degree})$	Scan incoming roads for max Q
Manager::tick() — full simulation step	$O(N * V^2 + E)$	$O(N * V^2)$	Dominated by rerouting cost

calcObjective() — objective function	$O(E)$	$O(E)$	One pass over all roads
updateMetrics() — performance metrics	$O(N + E)$	$O(N + E)$	Passes over vehicles and roads
Graph::shortestPath() — Dijkstra	$O(V^2)$	$O(V^2)$	For 36 nodes: extremely fast
Graph::updateEdgeWeight()	$O(\text{degree}(v))$	$O(1)$ avg	Search adjacency list of one node
Queue::enqueue/dequeue	$O(1)$	$O(1)$	Linked list operations
buildCity() — Initializer	$O(V + E)$	$O(V + E)$	One-time setup

For the 6x6 city grid: $V = 36$, $E = 120$. With N vehicles, rerouting all vehicles per tick costs $O(N * 36^2) = O(1296N)$. At typical $N < 50$, this is approximately 64,800 operations per tick, which runs comfortably within 100ms. The bottleneck is rerouteVehicles(), which scales linearly with vehicle count. The system degrades gracefully as N increases.

10. Implementation Details

10.1 Programming Language and Tools

Language	C++ (native) with C++/CLI for Windows Forms
IDE	Visual Studio 2022
GUI Framework	Windows Forms (C++/CLI)
Threading	System::Threading::Thread (background simulation thread)
Graphics	GDI+ via System::Drawing (double-buffered panel)
Graph ADT	Custom adjacency list (AdjList<T>)
Queue ADT	Custom linked-list queue (Queue<T>)

10.2 Project File Structure

All header files. Backend is header-only for simplicity and compatibility with C++/CLI:

File	Purpose
Observer.h	Pure abstract base interface for all observers
Subject.h	Base class managing observer list with STL list
Vehicle.h	Struct holding all vehicle state data
Road.h	Directed edge with BPR formula, queue, signal logic
Intersection.h	Graph node with adaptive signal control

Graph.h	Adjacency list graph with Dijkstra and edge weight update
Queue.h	Custom linked-list queue ADT
Initializer.h	Builds predefined 6x6 city map and exposes to Manager
Manager.h	Drives tick loop, owns vehicles, implements Observer
GUIObserver.h	Native C++ Observer bridging to managed Windows Forms class
trafficSimulation.h	Windows Forms GUI with drawing, threading, and user input

10.3 Key Classes and Methods

Manager::tick(): The master simulation function called every 100ms. Executes 7 ordered steps: syncWeights, updateVehicles, autoAdd, intersection ticks, road ticks, updateMetrics, calcObjective.

Road::tick(): Updates discharge queue based on signal state and downstream capacity, recalculates congestion and BPR travel time, then notifies all observers.

Intersection::tick(): Counts down signal timer, triggers yellow phase transition, then switches green to the incoming road with maximum queue size.

Manager::addVehicle(): Runs Dijkstra from source to destination, places vehicle on the first road of the route, and adds to vehicle list.

Manager::update() [Observer]: Called when any road notifies — syncs all graph edge weights then reroutes all active vehicles.

trafficSimulation::simLoop(): Background thread function that calls manager->tick() and sleeps 100ms, with Monitor::Enter/Exit mutex protection.

11. Key Features

- Graph-based road modeling: Directed weighted graph with float edge weights representing real-time BPR travel times.
- Dynamic vehicle simulation: Vehicles spawn, move, queue, and arrive with full lifecycle management including a disappear timer after arrival.
- Congestion-aware routing: Dijkstra runs with updated edge weights every tick, automatically routing vehicles around congestion.
- Adaptive traffic signal control: Each intersection gives green to the road with the longest vehicle queue, with yellow phase transitions.
- Multi-source multi-destination: Vehicles are auto-spawned every 20 ticks with random source and destination. User can also manually add vehicles.
- Queue-based intersection management: Custom Queue<Vehicle*> ADT at each road's downstream end, discharged based on signal state and capacity constraints.
- Observer pattern: Full decoupling of backend simulation from GUI. Roads and Intersections notify Manager and GUI independently.
- Thread-safe simulation: Background thread for tick loop with Monitor mutex. GUI updates via BeginInvoke for cross-thread safety.
- Static layer caching: Roads and grass are drawn once to a cached Bitmap. Only vehicles and signals are redrawn each frame, eliminating flicker.

- Zoom and pan: + and - buttons adjust zoom level (0.5x to 3.0x). Right-click drag pans the map for exploring larger city grids.
- Interactive editing: Click any intersection or road to select and edit its properties (Tg, length, speed, capacity, discharge rate).

12. Results and Outputs

12.1 Description of Outputs

The system produces a live visual simulation of the 6x6 city grid with all roads, intersections, vehicles, and traffic signals rendered in real time. Performance metrics are displayed in the side panel and updated every tick.

12.2 GUI Screenshot

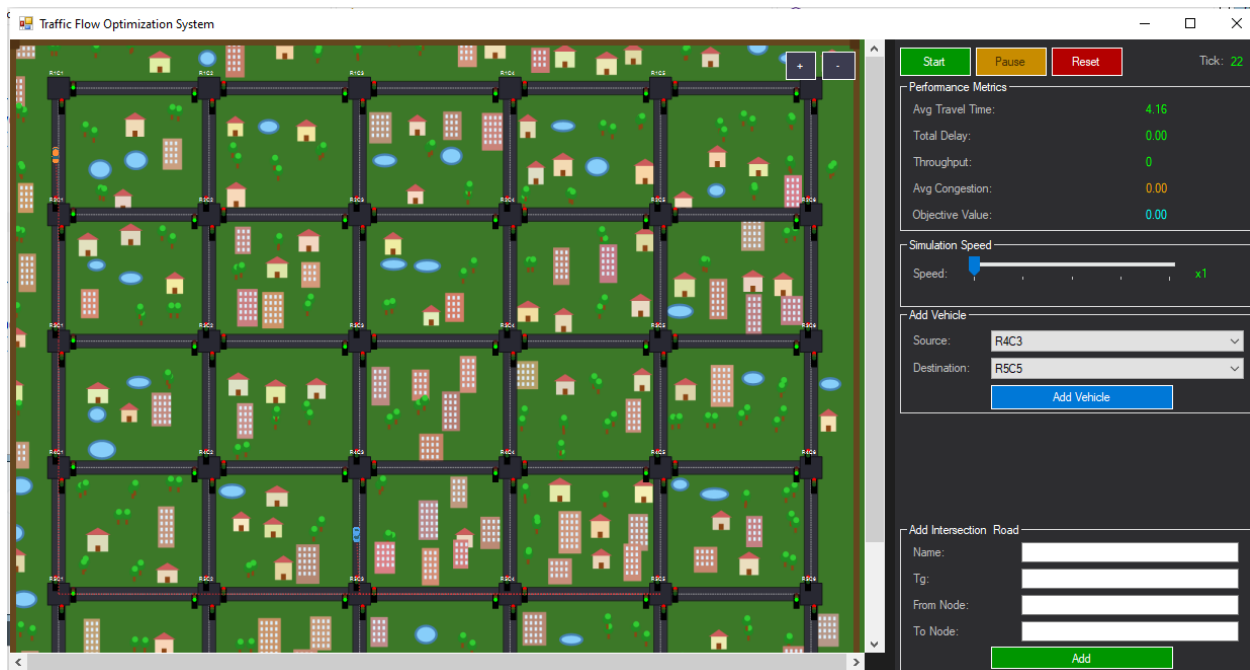


Figure 9.1: Live Simulation GUI

The screenshot shows the 6x6 city grid at tick 15. One vehicle (orange car) is visible moving along a road in the middle area. All intersections display traffic light indicators showing red/yellow/green states. The side panel shows performance metrics and the Add Vehicle panel. Zoom buttons are visible in the top-right of the city panel.

12.3 Observed Behavior

- Vehicles spawn every 20 ticks with random source and destination and immediately begin following the shortest path computed by Dijkstra.
- Roads change color from gray to orange to red as congestion increases.
- Traffic lights cycle through green, yellow, and red phases, always prioritizing the road with the longest queue.
- When a road becomes congested, active vehicles on other roads are automatically rerouted to avoid it.
- Arrived vehicles briefly remain visible before disappearing after their timer expires.

12.4 Performance Trends

- Average Travel Time increases as more vehicles are added and roads become congested.
- Total Delay measures the cumulative extra time vehicles spend compared to free-flow travel.
- Throughput (arrived vehicles) increases steadily over time as vehicles complete their routes.
- Objective Value decreases when signal control works effectively, then spikes as new vehicles are spawned.

13. Challenges and Solutions

13.1 Technical Challenges

Flow Corruption: Early implementation had f (vehicle count) going negative when vehicles were both removed from road body and also discharged from queue. Fixed by decrementing f in `vehicleFinished()` when vehicle leaves road, not again in `tick()`.

Circular Dependency: `Vehicle.h` needed `Road*` and `Road.h` needed `Vehicle*`. Solved using forward declaration of class `Road` in `Vehicle.h` and including only a pointer, avoiding full include.

Thread Safety: Background simulation thread modifying data while GUI thread reads it caused crashes. Solved by wrapping all `Manager::tick()` and GUI draw code with `Monitor::Enter/Exit` on a shared `simLock` object.

GUI Flicker: Redrawing all roads every frame caused visible flicker. Solved by building a static `Bitmap` cache of roads and grass, only invalidating it when road properties change.

13.2 Design Challenges

Queue Discharge Logic: The original discharge ignored downstream capacity, allowing vehicles to teleport into full roads. Fixed by checking `next->getF() < next->getC()` before discharging each vehicle.

Yellow Phase: Signal control without yellow phase caused instantaneous switching that looked unrealistic. Added a 3-tick yellow countdown in Intersection that blocks discharge during transition.

Observer Pattern Bridging: Connecting native C++ Observer to managed C++/CLI Windows Forms required `gcroot<trafficSimulation^>` in `GUIObserver` and `safe_cast` in `update()` implementation. This was placed after the full class declaration in `trafficSimulation.h`.

13.3 Implementation Issues

Graph Weight Type: Original Graph ADT used `int` weights. Since BPR travel times are float, all `pair<T, int>` in adjacency list had to be changed to `pair<T, float>`, requiring updates to `shortestPath()`, `insertEdge()`, and `MST` functions.

Incoming Road Preloading: Intersection needs to know its incoming roads for signal control, but the Graph manages edges. Solved by preloading incoming road pointers into each Intersection in `Manager` constructor.

14. Conclusion

14.1 Summary of Achievements

The Traffic Flow Optimization System was successfully designed and implemented as a complete C++ application. The system models a 6x6 city grid as a directed weighted graph, simulates vehicle movement with realistic congestion effects, implements adaptive signal control, and visualizes everything in a smooth Windows Forms GUI. All backend logic uses custom-implemented ADTs — a directed graph with Dijkstra, a linked-list queue, and the Observer/Subject pattern — with no reliance on STL for core data structures.

14.2 Objective Fulfillment

All primary and secondary objectives were met. The Observer pattern cleanly decouples the backend from the GUI. The BPR formula accurately models non-linear congestion effects. Dijkstra with dynamic edge weights achieves congestion-aware routing. Adaptive signal control prioritizes the most congested incoming road. Multi-threaded simulation with proper synchronization produces a smooth, flicker-free GUI. The objective function provides a quantitative measure of optimization performance.

14.3 Learning Outcomes

- Practical implementation of Graph ADT and Dijkstra's algorithm in C++.
- Application of Observer design pattern in a real multi-component system.
- Custom Queue ADT implementation for intersection management.
- Understanding of BPR traffic flow theory and discrete-time simulation models.

- Multi-threaded C++ programming with thread-safe GUI updates.
- Experience integrating native C++ with managed C++/CLI Windows Forms.

15. Future Enhancements

15.1 Possible System Extensions

- Priority Queue optimization for Dijkstra: Replace the current $O(V^2)$ linear scan with a min-heap based priority queue to achieve $O((V + E) \log V)$ complexity for large city grids.
- A* Algorithm: Replace Dijkstra with A* using Euclidean distance as the heuristic for faster routing in larger maps.
- Incident simulation: Add random road closures or accidents that force mass rerouting of affected vehicles.

15.2 Additional Features

- 3D visualization using OpenGL for a more realistic city appearance with building models.
- Vehicle types: trucks, buses, and motorcycles with different sizes, speeds, and capacities.
- Historical congestion data: Record and replay simulation states for post-analysis.
- Machine learning signal control: Replace the longest-queue heuristic with a trained reinforcement learning agent.

15.3 Real-World Deployment Possibilities

- Integration with real GPS and traffic sensor data feeds for live city traffic optimization.
- Smart city infrastructure: Connect to traffic light controllers for automated signal adjustment.
- Emergency vehicle routing: Priority lanes and signal preemption for ambulances and fire trucks.

16. References

Books

1. Thomas H. Cormen et al., Introduction to Algorithms (3rd Edition) — Dijkstra's Algorithm and Graph Theory.
2. Erich Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software — Observer Pattern.
3. Stanley B. Gershwin, Manufacturing Systems Engineering — BPR Traffic Flow Model.

Online Resources

1. Bureau of Public Roads (BPR) Volume-Delay Function — Transportation Research Board.

2. Microsoft C++/CLI Documentation — Threading and Windows Forms Integration.
3. DS Project Description Document — Traffic Flow Optimization System.
4. ChatGPT

Time Complexity

Variables used:-

- i) V = number of intersection (nodes)
- ii) E = number of roads (edges)
- iii) N = number of active vehicles
- iv) K = number of incoming roads at 1 inter.
- v) Q = queue size at a road.

1. Dijkstra's Algorithm:

This algorithm

works in two parts:

- i) Extract minimum from priority queue:

Each

extraction from queue costs $O(\log V)$.

We did it V times total:

$$V_{\text{extraction}} \times O(\log V)_{\text{each}} = O(V \log V)$$

- ii) Decrease edge:

Every edge is visited once each time we decrease that cost: $O(\log V)$

$$E_{\text{decreased}} \times O(\log V)_{\text{each}} = O(E \log V)$$

Total Dijkstra's

$$O(V \log V) + O(E \log V) \\ = O((V + E) \log V)$$

for a 6x6 grids:

$$V = 36 ; E = 120$$

DATE: / /

$$(36 + 120) \times \log_{12}(36) = 156 \times 5.1$$

$$= 806.5 \text{ operations}$$

* Dijkstra called:

Once per new vehicle added

$$O((V+E) \log V)$$

Once per active vehicle during rerouting:

$$O(N \times (V+E) \log V)$$

2. Queue

Operation:-

Queue is a linked list. Front and back pointers are maintained. All queue operations have same time complexity: $O(1)$

3. Road::tick() :-

- i) Discharge queue each $O(1)$
- ii) Update rho (congestion) $O(1)$
- iii) Update w (BPR formula) $O(1)$
- iv) Notify observer (2 observer) $O(1)$

$$\text{Per road} = O(1) + O(1) + O(1) + O(1)$$

$$\text{for all } E \text{ roads} = E \times O(1) = O(E)$$

4. Intersection::UpdateSignal() :-

DATE: / /

- i) Set all incoming roads to red $O(k)$
- ii) Scan k roads for longest queue $O(k)$
- iii) Set winner to green $O(1)$

Per instruction $= O(k) + O(k) + O(1)$
for all V instruction $= V \times O(k)$
 $= O(V \times k)$

5 Notify Observer:-

Each road has exactly 2 observers (Manager + GUT)
Loop runs 2 times $\rightarrow O(2)$ that means $O(1)$ per road

for all roads $= E \times O(1) = O(E)$

6 Sync Weights:-

Loop over all E roads. Update edge weight in graph
 $E \times O(1) = O(E)$

7 Update Vehicle:-

Loop over all N vehicles. for each:

Decrement $rv \rightarrow O(1)$

if $rv = 0$: enqueue to queue $O(1)$

pop route $O(1)$, assign next $O(1)$

All steps per vehicle $= O(1)$

$= N \times O(1)$

$= O(N)$

DATE: / /

8. Reroute Vehicle:-

Loop over all N vehicles. Run dijkstra for each one:

$$(N \times O(V+E) \log V)$$

$$= O(N \times (V+E) \times \log V)$$

9. Record Metric:-

Average Travel time $\rightarrow$ loop N vehicle $\rightarrow O(N)$

Total delay $\rightarrow$ loop $\rightarrow O(N)$

Throughput $\rightarrow$ running counter $\rightarrow O(1)$

Average congestion $\rightarrow$ loop road $\rightarrow O(E)$

$$O(N) + O(E) + O(1) + O(N)$$

$$= O(N+E)$$

10. Calculate Objective():-

Loop over all E roads. for each compute $0.5 \times Q.size() + 0.5 \times rho^2 \rightarrow O(1)$

for all roads $\Rightarrow E \times O(1) = O(E)$

11. Initialization (runs Once):-

each call
Scans V existing vertex to avoid new

$$V \times O(V) = O(V^2)$$

Similar for edges

$$E \times O(E) = O(E^2)$$

$$\text{Total} = O(V^2) + O(E^2) = O(V^2 + E^2)$$

DATE: / /

Full tick Summary:-

All those

operations that are working as
per ticks

Adding them all:

$$\begin{aligned} &O(E) + O(N) + O(V \times K) + O(E) + O(N+E) \\ &+ O(E) \\ &= O(N + E + V \times K) \end{aligned}$$

When routing is triggered (via observer)

$$O(N \times (V + E) \times \log V)$$